Mathématiques en Python

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. La librairie Math **3**

A. Les constantes mathématiques 3

B. Les fonctions mathématiques 4

C. Exercice : Quiz 6

III. La librairie NumPy **8**

A. Les matrices/tableaux 8

B. Manipulation de matrices 10

C. Exercice : Quiz 14

IV. La librairie Matplotlib **15**

A. Tracer une courbe avec Matplotlib 15

B. Exercice : Quiz 21

V. L'essentiel **22**

VI. Auto-évaluation **23**

A. Exercice 23

B. Test 23

Solutions des exercices **24**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 90

Environnement de travail : Python 3.12+, PyCharm Community

Pré-requis : bases en Python, bases en mathématiques (matrices, graphiques)

Contexte

Un des atouts principaux du langage Python est l'existence de nombreuses librairies supplémentaires gratuites et très bien documentées.

Dans ce cours, nous allons nous limiter aux librairies qui permettent de faire des mathématiques. Pour un développeur web, les bibliothèques mathématiques telles que Math, Matplotlib et NumPy offrent des fonctionnalités essentielles pour la manipulation, l'analyse et la visualisation de données.

En combinant ces bibliothèques, vous pourrez effectuer des tâches telles que la génération de graphiques interactifs, la création de tableaux de bord dynamiques et la visualisation des tendances ou des modèles dans les données. Ces fonctionnalités sont particulièrement utiles dans les domaines comme l'analyse de données, la science des données, la finance, la visualisation de données géospatiales, et bien d'autres encore.

En d'autres termes, ces bibliothèques permettent d'exploiter la puissance des calculs numériques et des visualisations graphiques dans le but d'améliorer l'analyse et la présentation des données sur les sites web.

En outre, les bibliothèques abordées dans le cadre de ce cours sont énormément utilisées dans le domaine de l'intelligence artificielle, et c'est ce qui les rend si réputées aujourd'hui.

II. La librairie Math

A. Les constantes mathématiques

La bibliothèque Math

Cette bibliothèque est très utile pour résoudre des problèmes mathématiques dans divers domaines tels que les sciences, l'ingénierie, la finance, l'apprentissage automatique et bien d'autres encore. Elle offre une grande flexibilité et facilite l'écriture de code performant pour des calculs mathématiques précis.

La bibliothèque Math permet notamment d'accéder à des constantes mathématiques telles que pi (π), extrêmement utile dans des calculs d'aires et de périmètres par exemple, ou encore 'e', le nombre d'Euler qui correspond à l'exponentielle de 1.

Exemple

Soit un cercle de rayon R. Supposons que nous voulions calculer sa surface ainsi que son périmètre. Ceci est très facile grâce à la librairie Math :

```python
import math

R = 0.03
perimetre = 2*math.pi*R
surface = math.pi * R*R

print(f"Périmètre : {perimetre}, Surface : {surface}")
```

Comme vous avez pu le constater dans l'exemple ci-dessus, il faut toujours commencer par importer la librairie que l'on souhaite utiliser, et pour faire appel à une constante, il faut écrire « *math.* » suivi du nom de la constante (math.pi, math.e, etc.).

On retrouve également dans cette librairie la constante math.inf, qui représente l'infini. Cette valeur est positive, mais il est possible de la faire précéder d'un signe « - » afin de désigner l'infini du côté des nombres négatifs.

B. Les fonctions mathématiques

La bibliothèque Math est une bibliothèque qui fournit également des fonctions mathématiques pour effectuer des opérations courantes. Elle offre un large éventail de fonctions mathématiques, telles que les fonctions trigonométriques, logarithmiques, exponentielles, les fonctions de calcul des racines carrées, des puissances, des arrondis et bien plus encore. Nous allons étudier pas à pas les fonctions les plus utiles pour réaliser des calculs mathématiques.

Exemple **Factorielle**

L'exemple suivant calcule la factorielle d'un nombre entier. La fonction factorielle est définie par la relation mathématique suivante :

$n! = . (n - 1) . (n - 2) . \ldots . 2 . 1$

La bibliothèque Math comprend une fonction « *factorial* » qui prend en argument un entier n positif et qui renvoie sa factorielle :

```
1 import math
2
3 n = 5
4 fact = math.factorial(n)
5
6 print(fact)  # Affiche 120 (5*4*3*2*1)
```

Notez que cette fonction n'est définie que pour des nombres entiers positifs.

Le tableau suivant reprend quelques fonctions de base qui sont implémentées dans la librairie 'Math', ainsi que l'argument attendu de la fonction et son résultat :

Nom de la fonction	Argument(s)	Résultat
math.exp(x)	un réel x (int, float)	L'exponentielle de x (float)
math.log(x, n)	un réel x (int, float), un entier n (int)	Le logarithme de x en base n (usuellement 2 ou 10) (float)
math.pow(x,y)	un réel x (int, float), un réel y (int, float)	x à la puissance y (équivalent à x**y) (float)
math.sqrt(x)	un réel positif x (int, float)	La racine carrée (« *squareroot* » en anglais) de x (float)

Tous droits réservés à STUDI - Reproduction interdite

Complément

Si vous désirez aller plus loin, le lien suivant est la documentation officielle de la librairie 'Math'. Cette page reprend l'ensemble des fonctions qui peuvent être utilisées dans le cadre de cette librairie.

Number-theoretic and representation functions[1]

Exemple **Racine carrée d'une exponentielle**

Supposons que nous souhaitons cumuler deux fonctions : calculer l'exponentielle de 3 et en prendre la racine carrée :

$$\sqrt{e^3}$$

Nous pouvons utiliser la librairie Math comme ceci :

```
1 import math
2
3 exp = math.exp(3)
4 print(math.sqrt(exp))    # Affiche 4.4816890703380645
```

Fonctions trigonométriques

Les fonctions trigonométriques (cosinus, sinus, tangente, etc.) peuvent également s'avérer être très utiles lorsque le problème à résoudre fait intervenir de la géométrie. Elles sont répertoriées dans le tableau ci-dessous :

Nom de la fonction	Argument(s)	Résultat
math.cos(x)	un réel x (représentant un angle en radians)	Le cosinus de x
math.sin(x)	un réel x (représentant un angle en radians)	Le sinus de x
math.tan(x)	un réel x (représentant un angle en radians)	La tangente de x
math.acos(x)	un réel x	L'arccosinus de x
math.asin(x)	un réel x	L'arcsinus de x
math.atan(x)	un réel x	L'arctangente de x
math.dist(x,y)	2 points x et y	La distance euclidienne entre les 2 points
math.hypot(x)	un vecteur x(ses coordonnées)	La norme euclidienne du vecteur x
math.cosh(x)	un réel x	Le cosinus hyperbolique de x
math.sinh(x)	un réel x	Le sinus hyperbolique de x

1 https://docs.python.org/3/library/math.html#number-theoretic-and-representation-functions

Tous droits réservés à STUDI - Reproduction interdite

Nom de la fonction	Argument(s)	Résultat
math.tanh(x)	un réel x	La tangente hyperbolique de x
math.acosh(x)	un réel x	L'arccosinus hyperbolique de x
math.asinh(x)	un réel x	L'arcsinus hyperbolique de x
math.atanh(x)	un réel x	L'arctangente hyperbolique de x

Rappel **Fonctions trigonométriques : quelques rappels**

La fonction acos, asin et atan sont les fonctions arc-cosinus, arc-sinus et arc-tangeante. Elles sont les fonctions réciproques de cos, sin et tan. C'est-à-dire qu'il existe les relations mathématiques suivantes :

$$acos(cos(x)) = x$$
$$asin(sin(x)) = x$$
$$atan(tan(x)) = x(x > -\pi/2 \text{ et } x < \pi/2)$$

Par ailleurs, une distance (euclidienne) entre deux points (x1,y1) et (x2,y2) dans un plan à 2 dimensions est donnée par :

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Finalement, la norme euclidienne d'un vecteur est donnée par :

$$n = \sqrt{x^2 + y^2}$$

À noter que la norme euclidienne d'un vecteur est identique au calcul de l'hypoténuse d'un triangle rectangle dont les deux petits cotés ont des longueurs x et y.

La bibliothèque 'Math' permet également de convertir des degrés en radians et vice-versa. Ceci peut se faire à l'aide des fonctions math.radians(x) et math.degrees(x).

C. Exercice : Quiz

[solution n°1 p.25]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

Il faut écrire « *import math* » avant chaque ligne où l'on souhaite utiliser une constante ou une fonction de cette librairie.

- ○ Vrai
- ○ Faux, c'est inutile
- ○ Faux, cela est inutile et va générer une erreur

Question 2

Lesquelles de ces fonctions font partie de la librairie Math ?

- ☐ tan(x)
- ☐ arctan(x)
- ☐ tanh(x)

Question 3

Lorsque j'utilise la fonction cos(x), l'angle x est …

- ○ En degrés
- ○ En radians
- ○ Cela n'a pas d'importance

Question 4

Le code suivant est-il correct ?

```
import math
x = 6
print(exp(x))
```

- ○ Il y a une erreur à la 1ere ligne
- ○ Il y a une erreur à la 2eme ligne
- ○ Il y a une erreur à la 3eme ligne
- ○ Le code ne comporte aucune erreur

Question 5

Quelle valeur est obtenue après l'exécution de ce programme ?

```
import math
x = 9
print(math.sqrt(9))
```

- ○ 5
- ○ 9 !
- ○ 3
- ○ Ce programme comporte une erreur

Tous droits réservés à STUDI - Reproduction interdite

III. La librairie NumPy

A. Les matrices/tableaux

Définition **Librairie NumPy**

La bibliothèque NumPy fournit des structures de données de tableaux multidimensionnels et des fonctions mathématiques pour effectuer des calculs rapides et efficaces sur ces tableaux. Elle permet de réaliser des opérations de manipulation de vecteurs et de matrices, ainsi que la résolution d'équations mathématiques complexes. Cette bibliothèque puissante facilite le traitement des données et offre des performances optimisées pour les calculs numériques.

Exemple **Créer une matrice**

Dans cet exemple, nous définissons une matrice à deux dimensions. La première ligne contiendra 1,2,3, tandis que la deuxième ligne contiendra 4,5,6 :

```
1 import numpy as np
2
3 array = np.array([[1,2,3], [4,5,6]])
4
5 print(array)
6
7 # Affiche :
8 #[[1 2 3]
9 # [4 5 6]]
```

Nous obtenons bien une matrice (2x3), c'est-à-dire une matrice contenant 2 lignes et 3 colonnes.

Remarque

Dans l'exemple précédent, vous aurez remarqué que nous importons la librairie NumPy en écrivant « *import… as …* ».

En effet, certains noms de librairies étant plus longs, il est parfois dommage de devoir écrire le nom de la librairie à chaque fois qu'on souhaite accéder à son contenu. La syntaxe « *import as* » permet de pouvoir faire appel au raccourci qui suit le « *as* » dans le code, lors de la sollicitation de la librairie. Dans l'exemple précédent, une fois l'import fait, on écrira donc « *np.array()* » au lieu de « *numpy.array()* ».

Exemple **Initialisation de matrices**

Initialiser une matrice peut se faire de différentes façons, les plus courantes étant :

- Initialiser avec des valeurs connues : c'est l'exemple précédent
- Initialiser avec un contenu à 0
- Initialiser avec un contenu à 1
- Initialiser avec un contenu diagonal

Voyons comment cela fonctionne :

```
1 import numpy as np
2
3 arrayOnes = np.ones([4,5])
4
5 print(arrayOnes)
6
7 # Affiche :
8 #[[1. 1. 1. 1. 1.]
```


Tous droits réservés à STUDI - Reproduction interdite

```
# [1. 1. 1. 1. 1.]
# [1. 1. 1. 1. 1.]
# [1. 1. 1. 1. 1.]]

arrayZeroes = np.zeros([4,5])

print(arrayZeroes)

# Affiche :
#[[0. 0. 0. 0. 0.]
# [0. 0. 0. 0. 0.]
# [0. 0. 0. 0. 0.]
# [0. 0. 0. 0. 0.]]

arrayEye = np.eye(4,4)

print(arrayEye)

# Affiche :
#[[1. 0. 0. 0.]
# [0. 1. 0. 0.]
# [0. 0. 1. 0.]
# [0. 0. 0. 1.]]
```

Initialisation de vecteurs

Parfois on souhaite définir des vecteurs (tableaux à une seule dimension) en définissant le début et la fin, et en laissant au programme le soin de définir les points intermédiaires. Les points intermédiaires seront distribués uniformément en fonction du nombre de points spécifié. Ceci est rendu possible par l'utilisation de la fonction numpy.linspace().

Exemple **fonction linspace()**

Définissons un vecteur de temps qui s'étend de 0 à 10, constitué de 20 points :

```
import numpy as np

timeseries = np.linspace(0, 10, 20)

print(timeseries)
```

Ceci nous donnera le vecteur suivant :

[0. 0.52631579 1.05263158 1.57894737 2.10526316 2.63157895
3.15789474 3.68421053 4.21052632 4.73684211 5.26315789 5.78947368
6.31578947 6.84210526 7.36842105 7.89473684 8.42105263 8.94736842
9.47368421 10.]

Qui correspond bien au vecteur demandé.

Tous droits réservés à STUDI - Reproduction interdite

Initialisation de vecteurs

Plutôt que de définir le nombre de points, on peut laisser au programme la liberté de le calculer, mais en lui spécifiant l'intervalle séparant deux points successifs. On peut alors utiliser la fonction numpy.arrange(), comme illustré dans l'exemple suivant :

Exemple **fonction linspace()**

Définissons un vecteur de temps qui s'étend de [0 à 10[(10 est exclu de l'intervalle), avec un pas de 1 :

```
1 import numpy as np
2
3 timeseries = np.arange(0, 10, 1)
4
5 print(timeseries)
```

Ceci nous donnera le vecteur suivant :

```
[0 1 2 3 4 5 6 7 8 9]
```

Qui correspond bien au vecteur demandé.

B. Manipulation de matrices

Récupérer les nombres dans un tableau

Pour commencer, Il est possible de consulter un élément d'un tableau 'tab' avec 'tab[i]', i correspondant à l'indice de l'élément dont on souhaite connaître la valeur. Pour un tableau à deux dimensions, il faut écrire 'tab[i][j]', *i* correspondant au numéro de la ligne et *j* au numéro de la colonne de l'élément qu'on souhaite récupérer.

Exemple **Récupérer un élément d'un tableau**

```
1 import numpy as np
2
3 array = np.array([ [1, 2, 3], [4, 5, 6]])
4
5 print (array)
6
7 element = array[1][1]
8
9 print(element)
```

Dans cet exemple, nous définissons d'abord un tableau à 2 lignes et 3 colonnes :

```
[[1 2 3]
 [4 5 6]]
```

Ensuite, on récupère l'élément de la 2eme ligne, 2eme colonne → 5.

Attention **Définition des indices**

En Python, comme dans d'autres langages de programmation (c'est le cas par exemple en C++), les tableaux commencent toujours à l'indice 0. Le 1e élément d'un tableau est donc l'élément localisé à l'indice 0, le 2e element est localisé à l'indice 1, etc.

Tous droits réservés à STUDI - Reproduction interdite

Obtenir la taille d'un tableau

La taille (dimension) d'un tableau peut être obtenue en accédant à l'attribut 'shape' d'un tableau. Il existe également l'attribut 'size', mais attention, ce dernier donnera simplement le nombre d'éléments dans le tableau, sans spécifier le nombre de lignes ou de colonnes :

Exemple **Taille d'un tableau**

```
1 import numpy as np
2
3 array = np.array([ [1, 2, 3], [4, 5, 6]])
4
5 print(array.size)
6 print(array.shape)
```

L'exécution de ce code donnera le résultat suivant :

```
6
(2, 3)
```

En effet, notre tableau contient bien 6 éléments (size), et il s'agit bien d'un tableau de 2 lignes et 3 colonnes (shape).

Trier un tableau

La fonction numpy.sort() permet de trier le contenu d'un tableau dans l'ordre numérique.

Exemple **Taille d'un tableau**

```
1 import numpy as np
2
3 array = np.array([ [1, 7, 2], [5, 4, 6]])
4
5 print(np.sort(array))
```

L'exécution de ce code donnera le résultat suivant :

```
[[1 2 7]
[4 5 6]]
```

Vous remarquerez que le tableau est trié ligne par ligne.

Concaténation de tableaux

La fonction numpy.concatenate() permet de concaténer deux tableaux (les fusionner).

Exemple **Concaténation de deux tableaux**

```
1 import numpy as np
2
3 array   = np.array([[1, 7, 2], [5, 4, 6]])
4 array2  = np.array([[1, 2, 3], [4, 5, 6]])
5
6 print(np.concatenate((array, array2)))
```

L'exécution de ce code donnera le résultat suivant :

```
[[1 7 2]
 [5 4 6]
```

Tous droits réservés à STUDI - Reproduction interdite

```
 [1 2 3]
 [4 5 6]]
```

Vous constatez que nos deux tableaux sont fusionnés en un seul. Par défaut, la fonction concatenate fusionne les tableaux selon l'axe 0, c'est-à-dire dans le sens des lignes (1ere dimension). Si l'on veut spécifier explicitement l'axe de concaténation, il s'agira de le donner dans la fonction concatenate :

```
1 print(np.concatenate((array, array2),1))
```

Ce qui donnera :

```
[[1 7 2 1 2 3]
 [5 4 6 4 5 6]]
```

Scinder un tableau

Pour scinder un tableau, il suffit de définir un nouveau tableau qui contiendra uniquement un sous-ensemble du tableau à scinder. Pour sélectionner plusieurs éléments d'un tableau, on utilisera les deux points (' :'), comme dans l'exemple ci-dessous :

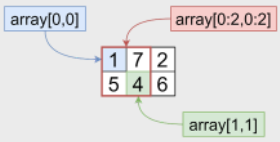
Exemple

```
1 import numpy as np
2
3 array   = np.array([[1, 7, 2], [5, 4, 6]])
4 print(array)
5 subArray = array[0:2, 0:2]
6
7 print(subArray)
```

L'exécution de ce code donnera le résultat suivant :

```
[[1 7 2]
 [5 4 6]]
[[1 7]
 [5 4]]
```

En effet, lorsque nous sélectionnons les indices 0 :2, cela signifie que l'on démarre la sélection à l'indice 0 (c'est-à-dire le 1e élément), et cette sélection s'arrête à l'indice 2 (c'est-à-dire le 3e élément, non inclus). Dans notre cas, cela signifie qu'on garde l'élément 1 et l'élément 2. On fait cette opération de sélection sur les lignes et sur les colonnes.



Multiplication matricielle

Il est possible de multiplier des tableaux/matrices par des nombres, mais également de multiplier des matrices entre elles (multiplication matricielle). Voyons ceci dans l'exemple qui suit.

Tous droits réservés à STUDI - Reproduction interdite

Exemple **Multiplication matricielle**

```
1 import numpy as np
2
3 array1 = np.array([[1, 7, 2], [5, 4, 6]])
4 array2 = np.array([[0, 1, 2], [4, 2, 9], [4, 6, 7]])
5 array3 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
6
7 print(array1)
8 print(array2)
9 print(array3)
10
11 print(np.matmul(array1, array2))
12
13 print(array2*array3)
```

L'exécution de ce code donnera le résultat suivant :

array1 :

```
[[1 7 2]
 [5 4 6]]
```

array2 :

```
[[0 1 2]
 [4 2 9]
 [4 6 7]]
```

array3 :

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

np.matmul(array1,array2) :

```
[[36 27 79]
 [40 49 88]]
```

array2*array3 :

```
[[ 0  2  6]
 [16 10 54]
 [28 48 63]]
```

La fonction matmul() permet de multiplier des matrices (multiplication matricielle classique). Tandis que l'opérateur de multiplication standard permet de multiplier les éléments deux par deux. Etant donné que la multiplication matricielle n'est définie que dans le cas où le nombre de colonnes de la 1ere matrice est égale au nombre de lignes de la seconde, nous avons défini une matrice array2 de dimensions (3x3).

Il existe également des fonctions qui permettent de réaliser des opérations classiques sur les tableaux, telles que :

- La recherche du maximum : tab.max() renvoie la valeur maximale contenue dans tab,
- La recherche du minimum : tab.min() renvoie la valeur minimale contenue dans tab,
- Sommer les éléments de la liste : tab.sum() renvoie la somme de toutes les valeurs contenues dans tab.

Tous droits réservés à STUDI - Reproduction interdite

Nous allons voir un exemple pratique de manipulation de cette bibliothèque dans la vidéo suivante :

Exemple **Multiplication matricielle**

```
1 import numpy as np
2
3 array1 = np.array([[1, 7, 2], [5, 4, 6]])
4
5 print(array1.max()) # Affiche 7
6 print(array1.min()) # Affiche 1
7 print(array1.sum()) # Affiche 25
```

C. Exercice : Quiz

[solution n°2 p.26]

Question 1

L'attribut shape d'un tableau représente :

- Le nombre d'éléments du tableau
- Le type de forme du tableau
- Les dimensions du tableau

Question 2

La fonction eye() de la librairie NumPy permet :

- De créer une matrice contenant uniquement des 0
- De créer une matrice contenant uniquement des 1
- De créer une matrice contenant des 0 partout, sauf sur la diagonale où elle contient des 1
- De voir le contenu d'une matrice

Question 3

Quelle(s) fonction(s) permet(tent) de créer un tableau dont les valeurs sont comprises entre 2 bornes et espacées d'un pas régulier que l'on choisit ?

- linspace
- arange
- concat
- matmul

Question 4

Ce code est-il correct ?

```
import numpy as np
m = np.array([ [1, 2 ,3], [4, 5, 6], [7,8,9], [10, 11] ])
```

- Oui
- Non : la fonction np.array() ne convient pas pour une matrice
- Non : il est impossible de créer cette matrice

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Quel tableau est créé après l'exécution de ce code ?

```
import numpy as np
t = np.arange(1, 20, 5)
```

- [1]
- [1, 6, 11, 16]
- [1, 5, 15, 20]

IV. La librairie Matplotlib

A. Tracer une courbe avec Matplotlib

Librairie Matplotlib

La librairie Matplotlib permet la création de graphiques et de visualisations attrayantes à partir des données traitées avec NumPy par exemple. Il y a donc un réel lien possible entre ces 2 bibliothèques. Cette bibliothèque visuelle offre une grande variété de graphiques, tels que les diagrammes en barres, les courbes, les diagrammes circulaires, les nuages de points et bien d'autres encore. Ces visualisations sont indispensables pour représenter les données de manière compréhensible et intuitive sur les sites web notamment.

Remarque

Dès que l'on souhaite utiliser Matplotlib, il faut réaliser l'import suivant :

```
import matplotlib.pyplot as plt
```

Et éventuellement :

```
import numpy as np
```

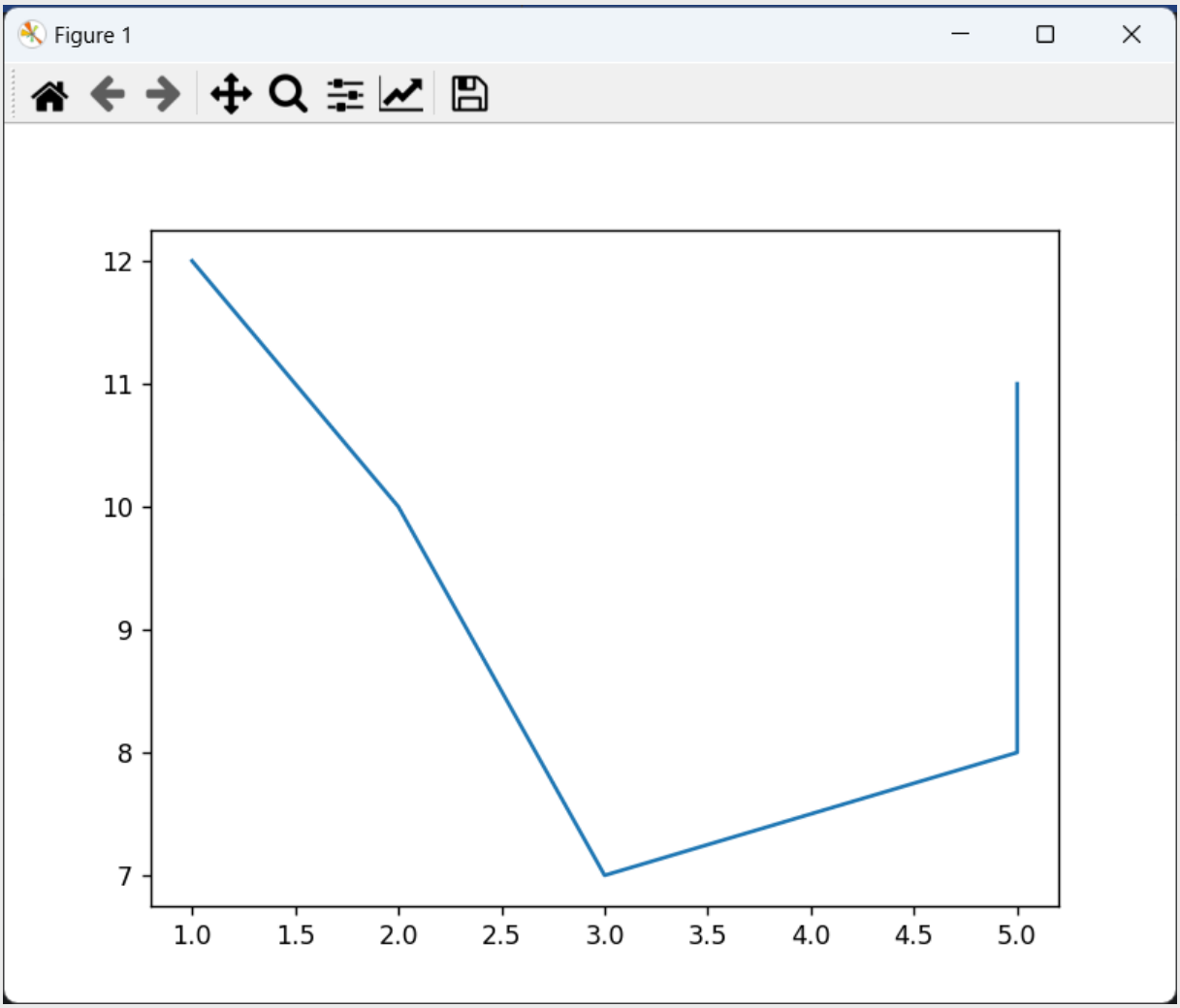
La méthode de base pour tracer une courbe est de faire appel à plt.plot(x, y), où x et y sont tous deux des tableaux qui peuvent être créés entres autres à l'aide de NumPy. Le tableau x contient les abscisses des points à placer, tandis que le tableau y contient leurs ordonnées. Ensuite, pour afficher le graphe, il suffit d'ajouter plt.show(). Voyons ceci avec un exemple concret.

Exemple **Tracer un graphe simple**

```
1 import matplotlib.pyplot as plt
2
3 t1 = [1, 2, 3, 5, 5]
4 t2 = [12, 10, 7, 8, 11]
5 plt.plot(t1, t2)
6 plt.show()
```

Tous droits réservés à STUDI - Reproduction interdite

L'exécution de ce code donnera le résultat suivant :



On constate que le graphique représente bien les points définis dans notre code, en abcysses les points 1, 2, 3, 5, 5, et en ordonnée les points 12, 10, 7, 8 et 11.

Afin d'améliorer l'esthétique du graphe, il est possible de changer la couleur en indiquant par exemple : 'green', 'blue', 'maroon', 'red'. Il est également possible de changer la ligne, pour par exemple la remplacer par des pointillés, en indiquant : linestyle = '--', ou linestyle = ':''. Ces options permettent notamment de bien distinguer 2 courbes superposées.

Exemple

Nous allons superposer les courbes de deux fonctions sur le même graphique. Les deux fonctions sont :

$$y1 = 2 . x$$

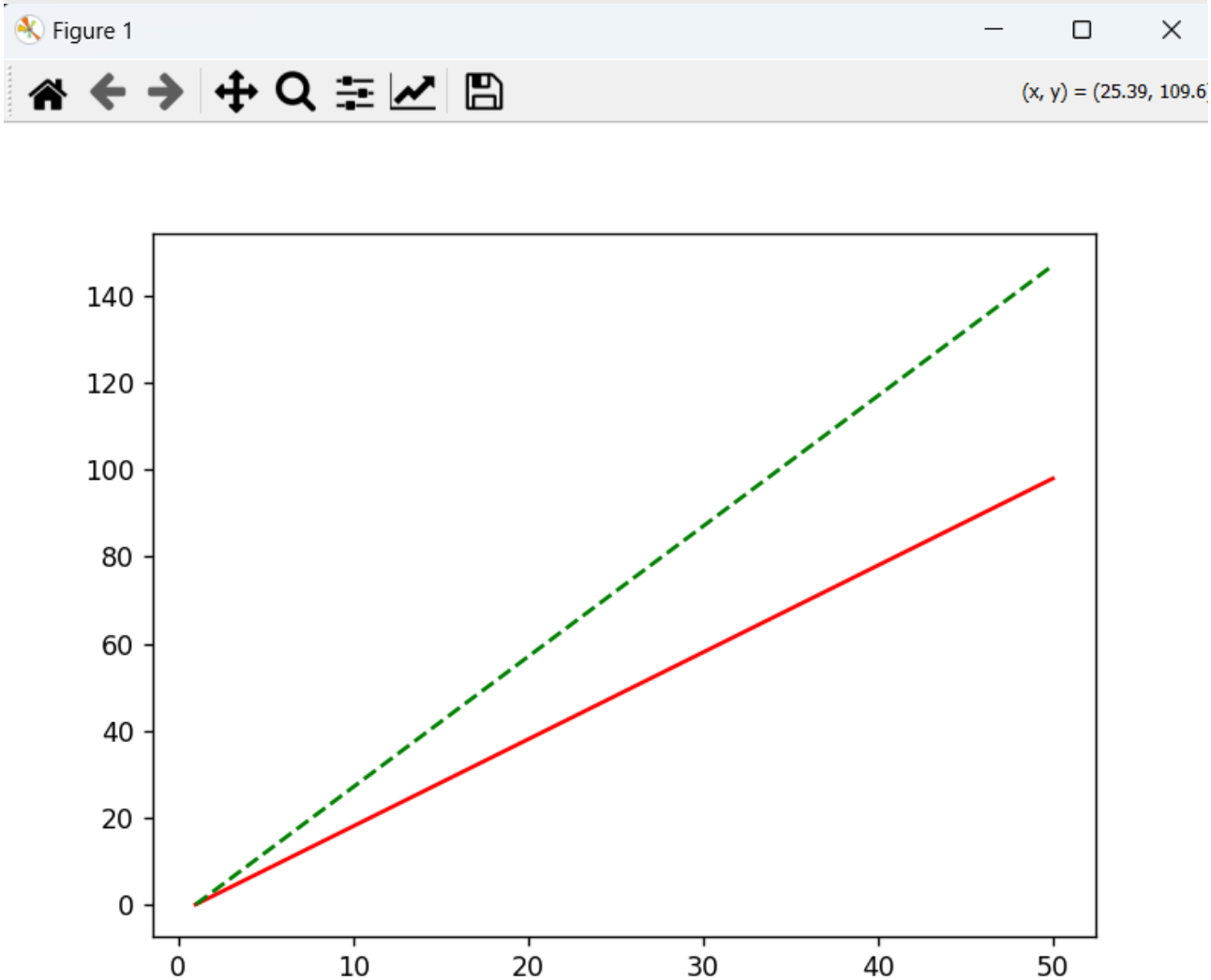
$$y2 = 3 . x$$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(1, 50, 50)
5 y1 = np.ones(50)
6 y2 = np.ones(50)
```


Tous droits réservés à STUDI - Reproduction interdite

```
7 for i in range(0, 50):
8     y1[i]=2*i
9     y2[i]=3*i
10
11 plt.plot(x, y1, 'red', linestyle = '-')
12 plt.plot(x, y2, 'green', linestyle = '--')
13
14 plt.show()
```

L'exécution de ce code donnera le résultat suivant :



Remarque

Pour superposer 2 courbes, il suffit donc d'écrire deux plt.plot() pour un seul plt.show(). Le fait de changer la couleur ou le style de la ligne aide à différencier les 2 courbes.

Tous droits réservés à STUDI - Reproduction interdite

Nous allons maintenant voir comment rendre notre graphique plus clair et plus informatif. Pour cela, nous pouvons commencer par donner un nom à chaque courbe afin de pouvoir les identifier à l'aide de label = '...'.

Ensuite, il est possible de créer une légende pour chaque axe avec plt.xlabel('...') pour les abscisses et plt.ylabel('...') pour les ordonnées.

Enfin, il est possible de donner un nom au graphe à l'aide de plt.title'('...'). À chaque fois, les pointillés entre guillemets représentent évidemment des strings.

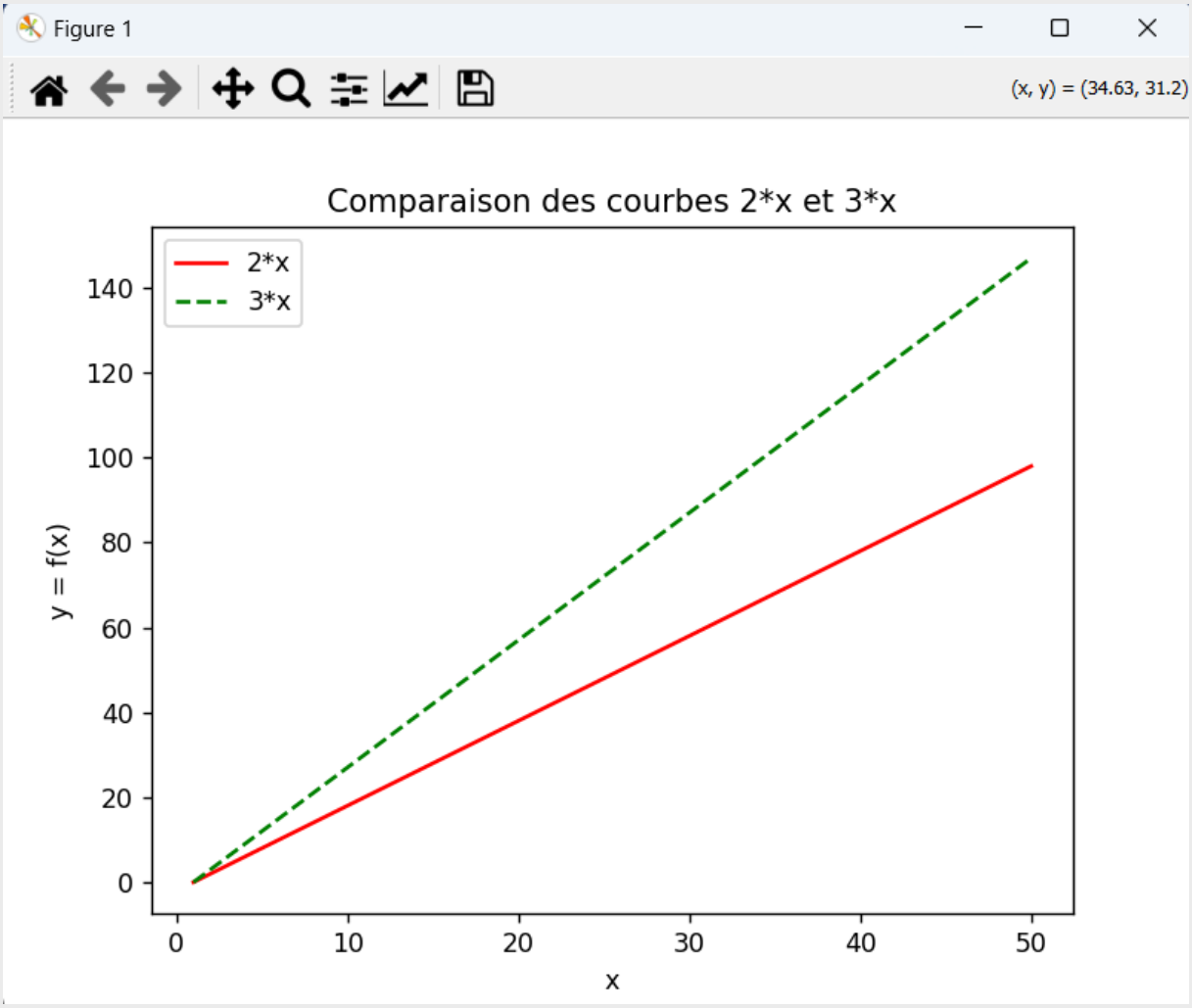
Exemple

Ajoutons une légende, des noms d'axes ainsi qu'un titre à notre graphique :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(1, 50, 50)
5 y1 = np.ones(50)
6 y2 = np.ones(50)
7 for i in range(0, 50):
8     y1[i]=2*i
9     y2[i]=3*i
10
11 plt.plot(x, y1, 'red', linestyle = '-')
12 plt.plot(x, y2, 'green', linestyle = '--')
13
14 plt.legend(["2*x","3*x"])
15 plt.xlabel("x")
16 plt.ylabel("y = f(x)")
17 plt.title("Comparaison des courbes 2*x et 3*x")
18
19 plt.show()
```


Tous droits réservés à STUDI - Reproduction interdite

Ce code donnera le résultat suivant :



Nous allons maintenant voir comment tracer un diagramme circulaire. Pour cela, il faut déclarer un tableau x contenant les différentes proportions du diagramme circulaire en pourcentage, puis avec un second tableau « *label* » qui contient des strings permettant de décrire chaque portion du diagramme circulaire.

Enfin, pour tracer, il faut utiliser la fonction plt.pie(x, labels=label) et plt.show() pour afficher le diagramme. Nous allons illustrer cette méthode dans l'exemple suivant :

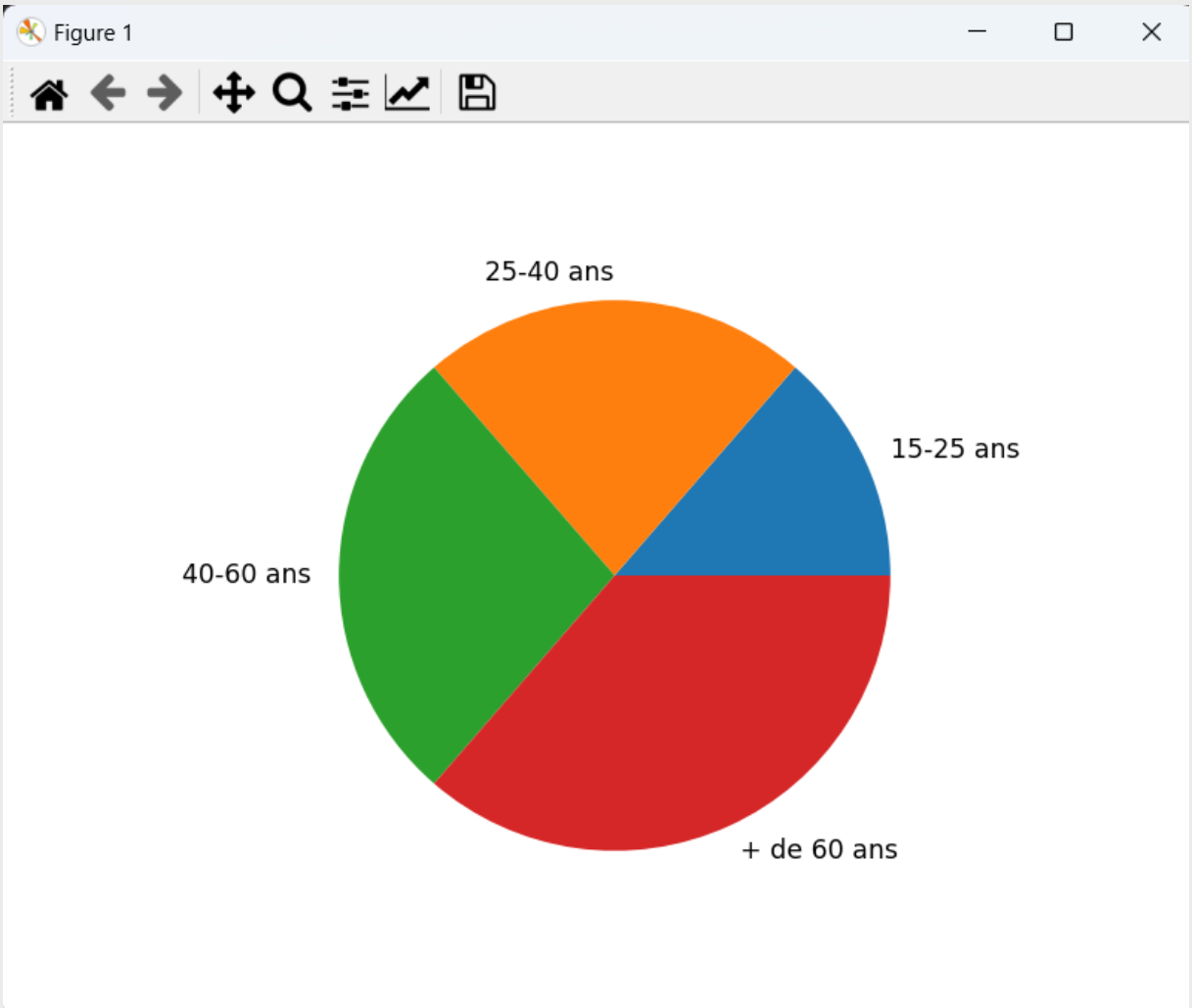
Exemple

Nous voulons tracer un diagramme circulaire afin de représenter les tranches d'âge d'utilisateurs d'un site web :

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = np.array([15, 25, 30, 40])
5 label = ["15-25 ans", "25-40 ans", "40-60 ans", "+ de 60 ans"]
6
7 plt.pie(x, labels=label)
8 plt.show()
```

Tous droits réservés à STUDI - Reproduction interdite

Ce code donnera le résultat suivant :



Enfin, nous allons voir comment tracer un diagramme en barres.

Il faut créer un tableau contenant les abscisses des différentes barres, et un autre tableau représentant les proportions correspondantes.

Enfin, pour tracer le diagramme en barres, il faut utiliser plt.bar(x,y). Nous allons illustrer cette méthode dans l'exemple suivant :

Exemple

Imaginons que l'on doive tracer un diagramme en barres permettant de représenter la répartition des notes d'élèves à un examen :

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = np.array(['0-5', '5-10', '10-15', '15-20'])
5 y = np.array([3, 7, 15, 5])
6
7 plt.bar(x,y)
8
9 plt.xlabel("Notes")
```


Tous droits réservés à STUDI - Reproduction interdite

```
10 plt.ylabel("Nombre d'élèves")
11 plt.title("Répartition des notes d'examen")
12
13 plt.show()
```

Ce code donnera le résultat suivant :

Vous avez maintenant toutes les clés en main pour créer un graphe à l'aide de Matplotlib. Le meilleur moyen de maîtriser ces méthodes est de les mettre en application en créant vos propres graphiques, cela vous aidera à retenir les différentes étapes à suivre et les fonctions à utiliser.

B. Exercice : Quiz

[solution n°3 p.27]

Question 1

Le code suivant permet d'afficher un graphe :

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [11, 32, 65, 44, 17]
plt.plot(x,y)
```

Tous droits réservés à STUDI - Reproduction interdite

- ○ Vrai
- ○ Faux : il y a une erreur dans le code
- ○ Faux : il manque quelque chose dans le code

Question 2

Quelle est la fonction à utiliser pour légender l'axe des abscisses ?

- ○ plt.xlengend()
- ○ plt.xlabel()
- ○ plt.title()

Question 3

Quelle est la fonction à utiliser pour tracer un diagramme en barres ?

- ○ plt.plot()
- ○ plt.bar()
- ○ plt.pie()

Question 4

Lorsque l'on appelle plt.pie(x,y), à quoi correspond x ?

- ○ Les proportions du diagramme, exprimées comme des nombres ou des pourcentages
- ○ Les proportions du diagramme, exprimées comme des nombres compris entre 0 et 1
- ○ La description des portions du diagramme

Question 5

Quelle(s) fonction(s) doi(ven)t être suivie(s) de plt.show() ?

- ☐ plt.plot()
- ☐ plt.bar()
- ☐ plt.pie()
- ☐ plt.imshow()

V. L'essentiel

Dans ce cours, nous avons étudié les 3 bibliothèques principales qui permettent de faire des mathématiques en Python.

Nous avons vu que la librairie Math permet d'utiliser de nombreuses constantes et fonctions qui sont indispensables afin de réaliser des calculs mathématiques.

La librairie NumPy, quant à elle, propose une grande diversité de fonctions qui permettent de créer et de manipuler les tableaux de façon très simple, rapide et efficace.

Enfin, la librairie Matplotlib est consacrée à la réalisation de graphiques et possède de nombreuses fonctions qui permettent d'améliorer la clarté et l'esthétique de vos visuels.

Tous droits réservés à STUDI - Reproduction interdite

Pour finir, il est important de garder en tête que bien souvent, ces 3 bibliothèques sont utilisées ensemble dans le but de générer un graphique. Ces librairies gratuites font partie des atouts majeurs du langage Python, car elles créent beaucoup de possibilités tout en étant très simples à utiliser, et très bien documentées sur Internet.

VI. Auto-évaluation

A. Exercice

Question 1

[solution n°4 p.29]

À l'aide de la librairie NumPy, créez des tableaux permettant de stocker les recettes effectuées chaque jour du mois. Voici les résultats de janvier en Euros :

[1100, 1231, 1452, 7680, 3542, 8765, 4567, 4536, 2343 ,9273, 1543, 7653, 7654, 2354, 7658, 1231, 4321, 1766, 9877, 9876, 3456, 3545, 2343, 1224, 5436, 2343, 6754, 7234, 2654, 3467]

Calculez les recettes minimales, maximales et moyennes à l'aide de NumPy.

Question 2

[solution n°5 p.29]

Tracez un graphe qui permettra de superposer les recettes quotidiennes effectuées au cours des 2 mois précédents à l'aide de Matplotlib.

Les résultats du mois antérieur (décembre) sont donnés ici :

[1766, 1543, 8765, 4536, 1100, 4567, 9877, 3467, 2343, 2654, 9273, 1231, 7654, 9876, 6754, 1224, 7680, 7658, 2343, 7653, 7234, 3545, 2343, 3542, 1231, 2354, 1452, 3456, 5436, 4321].

B. Test

Exercice 1 : Quiz

[solution n°6 p.30]

Question 1

Quel est le rôle de la bibliothèque NumPy ?

- ○ Utiliser des fonctions mathématiques
- ○ Tracer des courbes
- ○ Créer et utiliser des tableaux

Question 2

Quel est le signe de la constante math.inf ?

- ○ Positif
- ○ Négatif
- ○ Il n'a pas de signe

Question 3

Quel est le premier argument que prend la fonction plt.plot() ?

- ○ Les ordonnées des points à placer sur le graphe
- ○ Les coordonnées des points à placer sur le graphe
- ○ Les abscisses des points à placer sur le graphe

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Que calcule la fonction suivante : math.pow(x,y) ?

- x à la puissance y
- Le produit de x et y
- y à la puissance x

Question 5

Quel est le rôle de la fonction np.sort() ?

- Permuter les éléments d'un tableau
- Concaténer 2 tableaux
- Trier les éléments d'un tableau dans l'ordre croissant

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

studi

Exercice p. 6 **Solution n°1**

Question 1

Il faut écrire « *import math* » avant chaque ligne où l'on souhaite utiliser une constante ou une fonction de cette librairie.

- ○ Vrai
- ◉ Faux, c'est inutile
- ○ Faux, cela est inutile et va générer une erreur

Il est inutile d'importer plusieurs fois le même module dans un même programme. Il suffit de le faire une seule fois, habituellement au tout début du code.

Question 2

Lesquelles de ces fonctions font partie de la librairie Math ?

- ☑ tan(x)
- ☐ arctan(x)
- ☑ tanh(x)

Attention à la syntaxe, il ne faut pas écrire arctan(x) mais atan(x). Les 2 autres propositions sont tout à fait correctes.

Question 3

Lorsque j'utilise la fonction cos(x), l'angle x est …

- ○ En degrés
- ◉ En radians
- ○ Cela n'a pas d'importance

Par défaut, les fonctions trigonométriques de la librairie 'Math' travaillent toujours avec des radians. Il est donc important de s'assurer qu'on fournit 'x' en radians, et de faire une conversion le cas échéant avec la fonction math.radians().

Question 4

Le code suivant est-il correct ?

```
import math
x = 6
print(exp(x))
```

- ○ Il y a une erreur à la 1ere ligne
- ○ Il y a une erreur à la 2eme ligne
- ◉ Il y a une erreur à la 3eme ligne
- ○ Le code ne comporte aucune erreur

Tous droits réservés à STUDI - Reproduction interdite

Il faut penser à mentionner la librairie utilisée à chaque fois que l'on appelle une fonction ou une constante (math.exp(x) et non pas exp(x) directement ici), etc.

Question 5

Quelle valeur est obtenue après l'exécution de ce programme ?

```
import math
x = 9
print(math.sqrt(9))
```

- ○ 5
- ○ 9 !
- ◉ 3
- ○ Ce programme comporte une erreur

Ce code calcule la racine carrée du nombre 9, c'est donc 3.

Exercice p. 14 **Solution n°2**

Question 1

L'attribut shape d'un tableau représente :

- ○ Le nombre d'éléments du tableau
- ○ Le type de forme du tableau
- ◉ Les dimensions du tableau

L'attribut shape représente les dimensions d'un tableau. shape contiendra autant de nombres entiers qu'il y a de dimensions dans le tableau. Pour un tableau à 2 dimensions (matrices), shape contiendra le nombre de lignes et le nombre de colonnes du tableau.

Question 2

La fonction eye() de la librairie NumPy permet :

- ○ De créer une matrice contenant uniquement des 0
- ○ De créer une matrice contenant uniquement des 1
- ◉ De créer une matrice contenant des 0 partout, sauf sur la diagonale où elle contient des 1
- ○ De voir le contenu d'une matrice

La fonction eye() permet de créer ce que l'on appelle une matrice identité, c'est-à-dire une matrice comportant une diagonale de 1, et des 0 partout ailleurs.

Question 3

Quelle(s) fonction(s) permet(tent) de créer un tableau dont les valeurs sont comprises entre 2 bornes et espacées d'un pas régulier que l'on choisit ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ linspace
- ◉ arange
- ○ concat
- ○ matmul

Comme nous l'avons vu, 'arange' est une fonction qui permet de créer un tableau avec des valeurs comprises dans un intervalle, et toutes séparées d'un même pas. 'linspace' en revanche permet de choisir le nombre de valeurs contenues dans le tableau que l'on crée. La concaténation avec concat correspond à la fusion de 2 tableaux, tandis que matmul permet de multiplier deux matrices entre elles.

Question 4

Ce code est-il correct ?

```
import numpy as np
m = np.array([ [1, 2 ,3], [4, 5, 6], [7,8,9], [10, 11] ])
```

- ○ Oui
- ○ Non : la fonction np.array() ne convient pas pour une matrice
- ◉ Non : il est impossible de créer cette matrice

Attention, dans une matrice, tous les différents tableaux doivent avoir la même longueur (3 ici). Il faudrait donc rajouter un élément dans le dernier tableau pour que cette matrice puisse être créée. Elle contiendrait ainsi 4 lignes et 3 colonnes.

Question 5

Quel tableau est créé après l'exécution de ce code ?

```
import numpy as np
t = np.arange(1, 20, 5)
```

- ○ [1]
- ◉ [1, 6, 11, 16]
- ○ [1, 5, 15, 20]

La valeur de départ est 1. Ensuite, chaque valeur est obtenue en ajoutant 5 à la valeur précédente. Enfin, les valeurs du tableau doivent être strictement inférieures à 20, donc les valeurs du tableau s'arrêtent à 16.

Exercice p. 21 Solution n°3

Question 1

Le code suivant permet d'afficher un graphe :

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [11, 32, 65, 44, 17]
plt.plot(x,y)
```

Tous droits réservés à STUDI - Reproduction interdite

- ○ Vrai
- ○ Faux : il y a une erreur dans le code
- ◉ Faux : il manque quelque chose dans le code

Q Si vous n'écrivez pas 'plt.show()' à la fin de votre code, vous n'arriverez pas à afficher de graphe.

Question 2

Quelle est la fonction à utiliser pour légender l'axe des abscisses ?

- ○ plt.xlengend()
- ◉ plt.xlabel()
- ○ plt.title()

Q La seule fonction qui permet de créer une légende correspondant à l'axe des abscisses est plt.xlabel().

Question 3

Quelle est la fonction à utiliser pour tracer un diagramme en barres ?

- ○ plt.plot()
- ◉ plt.bar()
- ○ plt.pie()

Q En effet, plt.plot() trace une courbe et plt.pie() trace un diagramme circulaire (pie pour tarte, ce qui serait l'équivalent de ce que nous appelons un diagramme « *camembert* » en français). plt.bar() permet de tracer des graphiques en barres.

Question 4

Lorsque l'on appelle plt.pie(x,y), à quoi correspond x ?

- ◉ Les proportions du diagramme, exprimées comme des nombres ou des pourcentages
- ○ Les proportions du diagramme, exprimées comme des nombres compris entre 0 et 1
- ○ La description des portions du diagramme

Q x correspond au tableau des proportions, qui doivent être exprimées en pourcentages (donc 5, 10, 20 par exemple). En revanche, c'est y qui correspond à la description des portions du diagramme.

Question 5

Quelle(s) fonction(s) doi(ven)t être suivie(s) de plt.show() ?

- ☑ plt.plot()
- ☑ plt.bar()
- ☑ plt.pie()
- ☐ plt.imshow()

Q Ces 3 fonctions doivent toujours être suivies de plt.show() afin d'afficher les diagrammes que l'on souhaite tracer.

Tous droits réservés à STUDI - Reproduction interdite

La fonction plt.imshow() est utilisée pour afficher des images plutôt que des graphiques, et elle ne nécessite pas l'appel à plt.show().

p. 23 Solution n°4

```
import numpy as np

t1 = np.array( [1100, 1231, 1452, 7680, 3542, 8765, 4567, 4536, 2343, 9273, 1543, 7653, 7654, 2354, 7658, 1231, 4321,
                1766, 9877, 9876, 3456, 3545, 2343, 1224, 5436, 2343, 6754, 7234, 2654, 3467])

min = np.min(t1)
max = np.max(t1)
sum = np.sum(t1)

print(min)
print(max)
print(sum)
```

Le code affiche les éléments suivants :

```
1100
9877
136878
```

Qui correspondent respectivement au min, max et à la somme des éléments.

p. 23 Solution n°5

```
import numpy as np
import matplotlib.pyplot as plt

t1 = np.array( [1100, 1231, 1452, 7680, 3542, 8765, 4567, 4536, 2343, 9273, 1543, 7653, 7654, 2354, 7658, 1231, 4321,
                1766, 9877, 9876, 3456, 3545, 2343, 1224, 5436, 2343, 6754, 7234, 2654, 3467])

t2 = np.array([1766, 1543, 8765, 4536, 1100, 4567, 9877, 3467, 2343, 1231, 7654, 9876, 6754, 1224, 7680, 7658, 2343,
               7653, 7234, 3545, 1231, 2354, 1452, 3456, 5436, 4321])


time1 = np.arange(1, t1.size+1, 1)
time2 = np.arange(1, t2.size+1, 1)

plt.plot(time1, t1, 'red', linestyle = '-')
plt.plot(time2, t2, 'green', linestyle = '--')

plt.legend(["Janvier","Décembre"])
plt.xlabel("Jours")
plt.ylabel("Recettes")
plt.title("Recettes du mois de janvier et décembre")

plt.show()
```

Nous définissons d'abord nos deux vecteurs t1 et t2 qui représentent les recettes des mois de janvier et de décembre, respectivement.

Tous droits réservés à STUDI - Reproduction interdite

Ensuite, nous définissons le vecteur de temps associé à chaque vecteur de recettes. Pour cela, nous pouvons utiliser la fonction arange(), ainsi que l'attribut size, qui permet de récupérer la taille de chaque vecteur t1 et t2.

Pour terminer, nous traçons simplement les graphiques à l'aide de la fonction plot().

Quelques éléments graphiques sont ajoutés, comme une légende, des titres d'axes ainsi qu'un titre de graphique.

Exercice p. 23 Solution n°6

Question 1

Quel est le rôle de la bibliothèque NumPy ?

- ○ Utiliser des fonctions mathématiques
- ○ Tracer des courbes
- ◉ Créer et utiliser des tableaux

Comme nous l'avons vu, la bibliothèque NumPy permet de créer et de manipuler des tableaux. La première proposition correspond à la bibliothèque math et la deuxième proposition correspond à la bibliothèque Matplotlib.

Question 2

Quel est le signe de la constante math.inf ?

- ◉ Positif
- ○ Négatif
- ○ Il n'a pas de signe

La constante math.inf représente l'infini positif et stocke en réalité le plus grand nombre positif que l'on peut utiliser en Python. Pour utiliser l'infini négatif, il suffit d'écrire « *-math.inf* ».

Question 3

Quel est le premier argument que prend la fonction plt.plot() ?

- ○ Les ordonnées des points à placer sur le graphe
- ○ Les coordonnées des points à placer sur le graphe
- ◉ Les abscisses des points à placer sur le graphe

Comme nous l'avons vu dans le cours, il faut d'abord renseigner les abscisses des points à placer sur le graphe, et ensuite leurs ordonnées, sous forme de tableaux.

Question 4

Que calcule la fonction suivante : math.pow(x,y) ?

- ◉ x à la puissance y
- ○ Le produit de x et y
- ○ y à la puissance x

x est le nombre à élever à la puissance y, l'ordre des arguments est ici primordial.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Quel est le rôle de la fonction np.sort() ?

- ○ Permuter les éléments d'un tableau
- ○ Concaténer 2 tableaux
- ◉ Trier les éléments d'un tableau dans l'ordre croissant

En effet, « *to sort* » signifie trier, donc le rôle de cette fonction est bien de trier les éléments, en l'occurrence dans l'ordre croissant.

Tous droits réservés à STUDI - Reproduction interdite